

Tecnológico Nacional de México
Instituto Tecnológico de Mazatlán

Ingeniería en Sistemas Computacionales

Reporte de práctica

Control de servomotor por interfaz web

Arduino UNO R4 WiFi · Servomotor MG996R

Materia	Sistemas Programables
Práctica	Control de servomotor por interfaz web
Número de control	L231000103
Placa	Arduino UNO R4 WiFi
Herramientas	Arduino IDE 2, Fritzing, Tinkercad Circuits, navegador web
Fecha de pruebas	14 de septiembre de 2026
Fecha de entrega	16 de septiembre de 2026

Resumen

Se desarrolló un sistema que permite posicionar un servomotor MG996R desde una página web. El Arduino UNO R4 WiFi crea su propia red inalámbrica y funciona como servidor HTTP: al mover un control deslizante en el navegador, la página envía el ángulo deseado y el Arduino genera en el pin D9 la señal PWM correspondiente. En las pruebas el servo se desplazó a nueve ángulos entre 7° y 167°. El análisis del monitor serie reveló que la versión inicial del programa aplicaba cada ángulo dos veces; se identificó la causa, se corrigió el programa y se añadió el control en tiempo real que solicita la práctica.

I. Introducción

Los servomotores se usan cuando se necesita llevar un eje a una posición precisa, por ejemplo en brazos robóticos, cámaras orientables o compuertas automáticas. A diferencia de un motor de corriente directa común, el servo integra un motor, un tren de engranes, un potenciómetro y un circuito de control, por lo que basta con indicarle el ángulo deseado mediante una señal de pulsos.

Controlar un actuador desde un navegador web es la base de muchas aplicaciones de Internet de las Cosas: no requiere instalar una aplicación, funciona en cualquier teléfono o computadora y permite operar el sistema a distancia. El Arduino UNO R4 WiFi facilita este tipo de proyectos porque incluye un módulo inalámbrico en la misma placa. En esta práctica se integraron ambos elementos para enviar el ángulo desde una página web y posicionar el servomotor en consecuencia.

II. Objetivos

Objetivo general. Controlar la posición de un servomotor MG996R desde una interfaz web servida por el Arduino UNO R4 WiFi, de modo que el servo siga en tiempo real el ángulo seleccionado por el usuario.

- Configurar el Arduino UNO R4 WiFi como punto de acceso y servidor HTTP.
- Diseñar una página web con un control deslizante de 0° a 180° que envíe el ángulo sin recargar la página.
- Generar la señal PWM del servo con la librería Servo y validar que el ángulo recibido esté dentro del rango permitido.
- Conectar el servo con fuente externa y tierra común, y documentar el circuito en Fritzing.
- Registrar y analizar el comportamiento del sistema mediante el monitor serie.

III. Marco teórico

A. Servomotor y modulación por ancho de pulso

Un servomotor de posición recibe una señal periódica de 50 Hz, es decir, un pulso cada 20 ms. La duración de ese pulso indica el ángulo: en la librería Servo de Arduino, 544 μ s corresponden a 0° y 2400 μ s a 180°, con una relación lineal entre ambos extremos [2]. El circuito interno del servo compara esa orden con la posición que mide su potenciómetro y mueve el motor hasta que coinciden.

El MG996R es un servo de engranes metálicos y alto par. Su hoja de datos indica un voltaje de operación de 4.8 a 7.2 V y una corriente que puede llegar a unos 2.5 A cuando el eje está bloqueado [4]. Por esa razón se recomienda alimentarlo con una fuente externa y unir su tierra con la del microcontrolador, para que ambos compartan la misma referencia de voltaje en la señal de control.

B. Arduino UNO R4 WiFi como servidor web

El UNO R4 WiFi combina un microcontrolador Renesas RA4M1 con un módulo ESP32-S3 que se encarga de la comunicación inalámbrica [1]. La librería WiFiS3 permite crear un punto de acceso con `WiFi.beginAP()` y atender conexiones con `WiFiServer` [3]. El navegador se comunica con el Arduino mediante el protocolo HTTP: envía una petición cuya primera línea contiene el método y la ruta (por ejemplo, `GET /servo?angle=90 HTTP/1.1`), seguida de varios encabezados, y el servidor responde con un código de estado y un contenido [6]. En la página, la interfaz `fetch()` de JavaScript envía peticiones sin recargar el documento, lo que permite actualizar el servo mientras se arrastra el control [5].

IV. Materiales y herramientas

Tabla I. Materiales y herramientas.

Elemento	Uso en la práctica
Arduino UNO R4 WiFi	Crea la red WiFi, sirve la página y genera la señal PWM en D9
Servomotor MG996R	Actuador que se posiciona según el ángulo recibido
Fuente externa de 4.8 a 6 V (4 pilas o eliminador de 5-6 V y 2 A o más)	Alimentación del servomotor
Protoboard y cables Dupont	Unión de la señal, la alimentación y la tierra común
Cable USB	Programación de la placa y monitor serie
Computadora o celular con navegador	Uso de la interfaz web
Arduino IDE 2 y librerías WiFiS3 y Servo	Programación y registro en el monitor serie
Fritzing	Diagramas de protoboard y esquemático
Tinkercad Circuits	Simulación inicial de las conexiones

V. Desarrollo

A. Circuito

El cable de señal del servo (naranja) se conecta al pin D9, que tiene salida PWM. El cable rojo (V+) y el café (GND) se conectan a los rieles de la protoboard, alimentados por la fuente externa. El GND del Arduino se une al riel negativo, de modo que el Arduino, el servo y la fuente comparten la misma tierra. El pin de 5 V del Arduino no se conecta al servo. La Figura 1 muestra la vista de protoboard y la Figura 2 el esquemático, ambos elaborados en Fritzing. Como Fritzing no incluye la pieza del UNO R4 WiFi, se usó la del UNO Rev3, que tiene el mismo formato y la misma distribución de pines.

Control de servomotor por interfaz web — diagrama de conexiones

Vista protoboard (Fritzing): Arduino UNO R4 WiFi + servomotor MG996R con fuente externa y tierra común

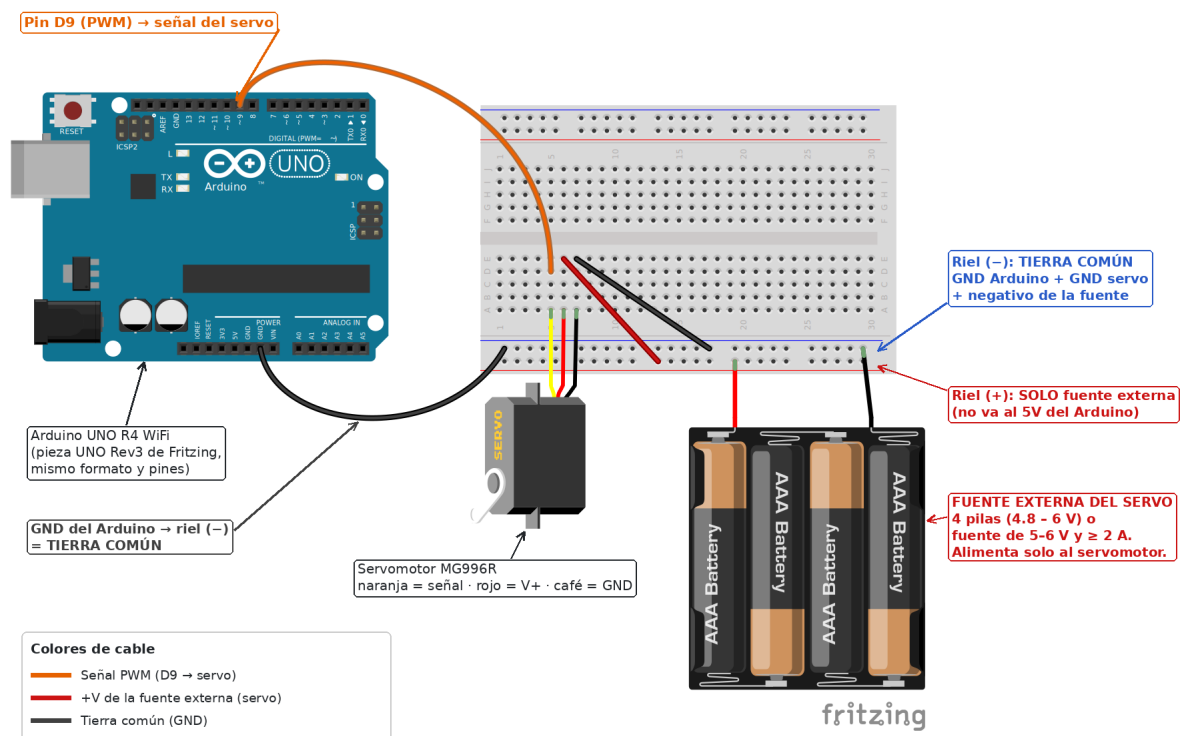


Figura 1. Diagrama de conexiones en vista protoboard (Fritzing).

Diagrama esquemático

Fritzing — la fuente externa alimenta al servo; las tres tierras están unidas

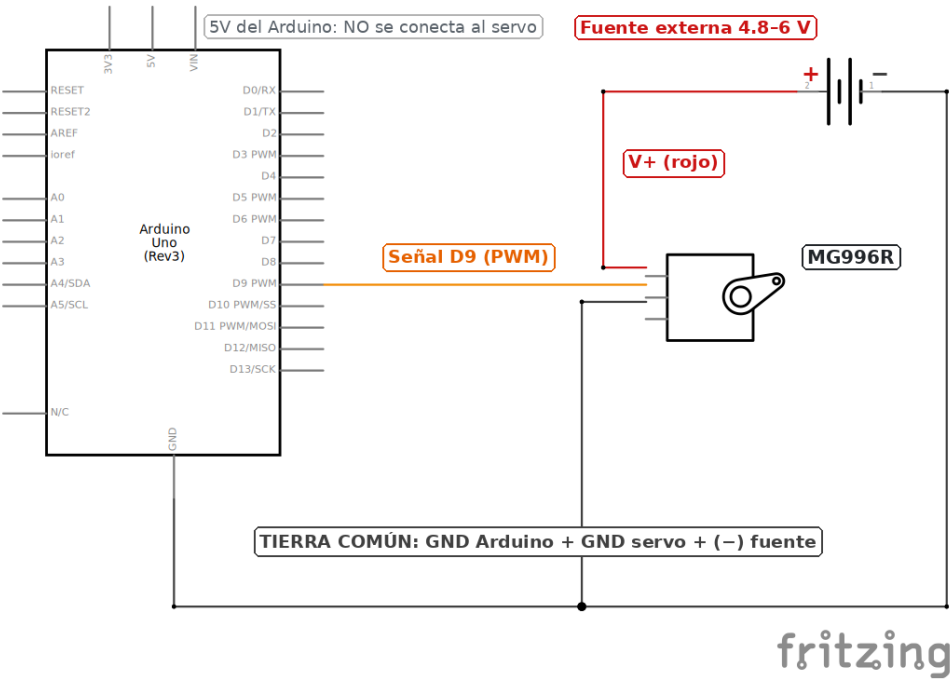


Figura 2. Diagrama esquemático (Fritzing).

Tabla II. Conexiones del circuito.

Servo MG996R	Conexión	Función
Naranja (señal)	Pin D9 del Arduino	Recibe el pulso PWM que indica el ángulo
Rojo (V+)	Positivo de la fuente externa	Alimenta el motor del servo
Café (GND)	Negativo de la fuente externa	Retorno de corriente del servo
—	GND del Arduino al negativo de la fuente	Tierra común entre placa, servo y fuente

B. Programa

Al iniciar, el programa coloca el servo en 90°, crea la red *Arduino-Servo* y arranca el servidor en el puerto 80. En cada ciclo revisa si hay un cliente conectado, lee la primera línea de la petición, descarta los encabezados y decide la acción según la ruta. El ángulo se acepta solo si tiene de uno a tres dígitos y está entre 0 y 180; el servo se mueve únicamente cuando el valor cambia, y cada cambio se registra en el monitor serie junto con el ancho de pulso aplicado. La Figura 3 resume el flujo de información y la Tabla III las rutas del servidor.

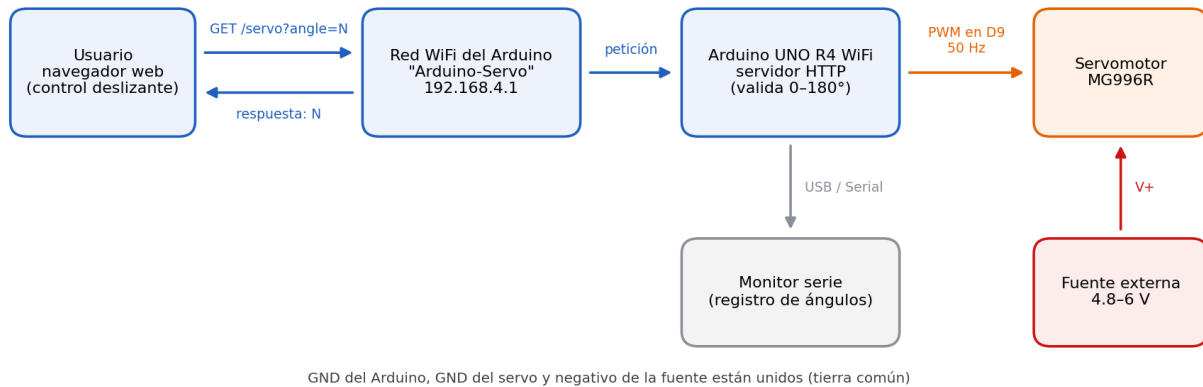


Figura 3. Diagrama de bloques del sistema.

Tabla III. Rutas del servidor web.

Ruta	Acción	Respuesta
GET /	Envía la página de control	200, HTML
GET /?angle=N	Mueve el servo y envía la página (formulario sin JavaScript)	200, HTML
GET /servo?angle=N	Mueve el servo en tiempo real	200 con el ángulo, o 400 si no es válido
GET /estado	Consulta el ángulo actual	200 con el ángulo
GET /favicon.ico	No realiza ninguna acción	204 sin contenido

C. Interfaz web

La página conserva el diseño de la primera versión: título, ángulo actual, control deslizante de 0° a 180° y botón *Mover Servo*. Al abrirse consulta `/estado` para mostrar la posición real del servo. Mientras el usuario arrastra el control, envía el ángulo a `/servo`; si todavía hay un envío en curso, guarda solo el valor más reciente para no saturar al Arduino. Una línea de estado indica si la comunicación funciona.

VI. Resultados

Al cargar el programa, el monitor serie mostró la creación de la red, la contraseña y la dirección 192.168.4.1. Desde la computadora conectada a esa red se abrió la página y se enviaron los ángulos 50°, 25°, 124°, 12°, 166°, 83°, 124°, 167° y 7°. La Figura 4 muestra la interfaz web funcionando después de enviar 7°; en la barra de direcciones se observa la petición `?angle=7`. La Figura 5 muestra el registro del monitor serie y la Figura 6 el armado físico.

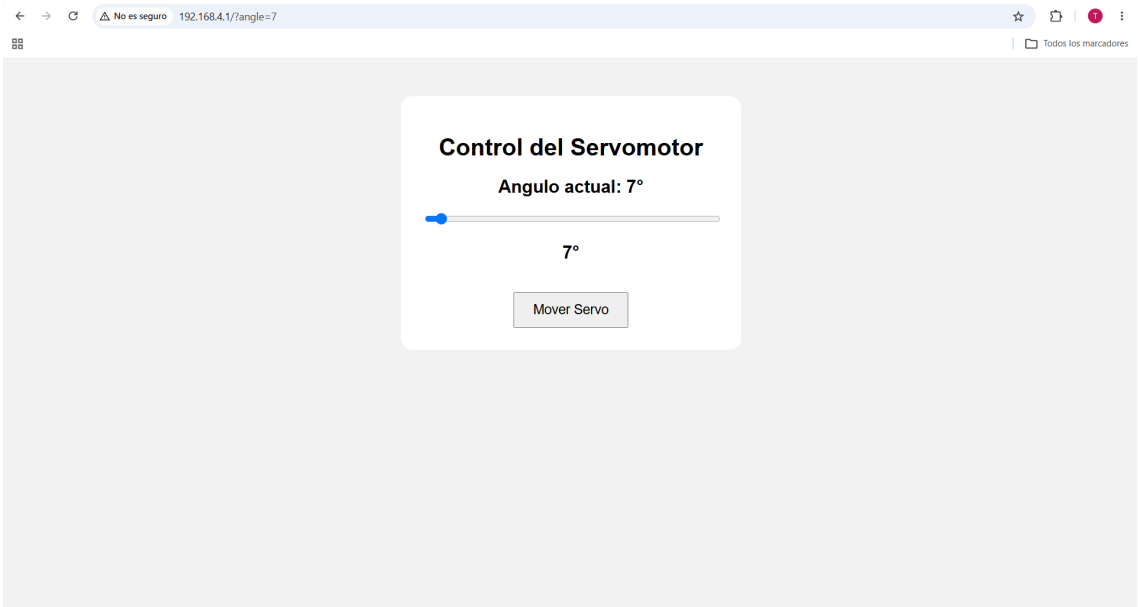


Figura 4. Interfaz web funcionando (ángulo actual: 7°).

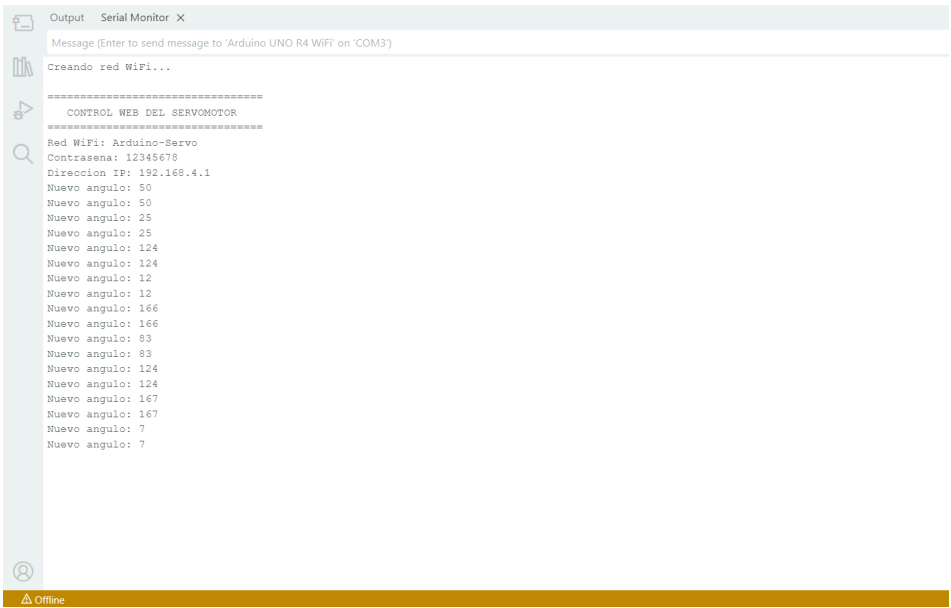


Figura 5. Monitor serie durante la prueba.

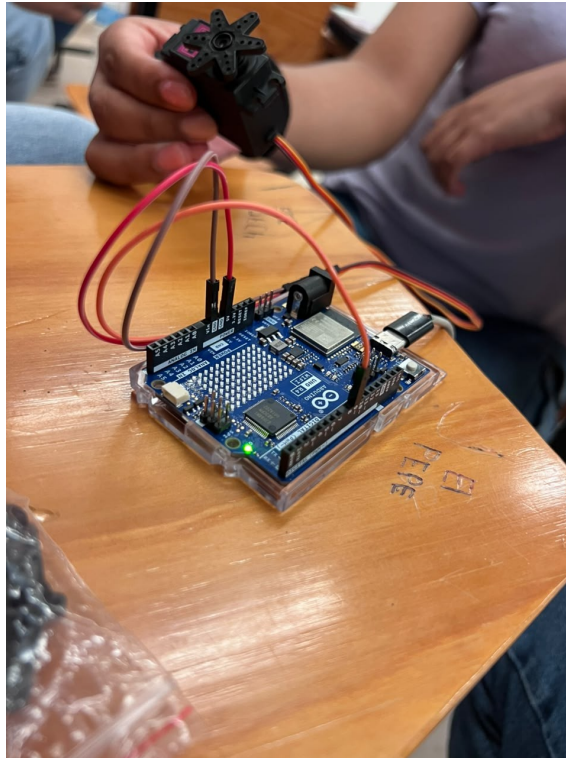


Figura 6. Armado físico con el Arduino UNO R4 WiFi y el servomotor MG996R.

Tabla IV. Ángulos enviados y señal PWM correspondiente.

Ángulo enviado	Pulso (μs)	Ciclo de trabajo	Veces en el monitor serie
50°	1059	5.29 %	2
25°	801	4.00 %	2
124°	1822	9.11 %	2
12°	667	3.33 %	2
166°	2255	11.28 %	2
83°	1399	7.00 %	2
124°	1822	9.11 %	2
167°	2265	11.32 %	2
7°	616	3.08 %	2

Las gráficas completas, la prueba de escritorio del programa corregido y el archivo de datos se encuentran en Resultados/Resultados.pdf y Resultados/datos_practica.csv. El video del funcionamiento está disponible en https://youtu.be/AtDGY__b80o.

VII. Análisis de resultados

Respuesta del sistema. El servo se posicionó en cada ángulo enviado y la página mostró el mismo valor que registró el Arduino, lo que confirma que la cadena navegador-WiFi-servidor-PWM funcionó correctamente. Los ángulos probados cubren casi todo el rango del servo y ambos sentidos de giro. Según la Tabla IV, el ancho de pulso varió entre 616 μ s y 2265 μ s, es decir, entre el 3.08 % y el 11.33 % del periodo de 20 ms.

Ángulos duplicados. Todos los valores aparecieron dos veces en el monitor serie (Figura 5). La causa es que el navegador, después de recibir la página, solicita el ícono /favicon.ico y en esa petición incluye el encabezado Referer con la dirección anterior, que contiene angle=. El programa inicial buscaba ese texto en toda la petición y no solo en la ruta, así que volvía a aplicar el mismo ángulo. El caso se reprodujo compilando el código original en la computadora con librerías simuladas: una petición de la página y otra del ícono generaron dos registros «Nuevo ángulo: 50». Aunque en este caso no movía el servo a otra posición, el error duplicaba trabajo y podría aplicar un ángulo que el usuario ya no desea. La versión corregida analiza únicamente la primera línea, responde al ícono con el código 204 y no vuelve a escribir en el servo si el ángulo no cambió.

Control en tiempo real. En la versión inicial el servo solo se movía al presionar el botón, y cada movimiento recargaba la página completa, lo que agregaba un retardo visible. Con la versión corregida, el control deslizante envía peticiones cortas mientras se arrastra y el Arduino responde solo con el número del ángulo. Para evitar que se acumulen peticiones, la página nunca mantiene más de un envío a la vez y descarta los valores intermedios. La lógica del servidor se verificó con una prueba de escritorio que incluyó ángulos válidos, valores fuera de rango, texto no numérico y rutas desconocidas; todos los casos dieron el resultado esperado. Falta repetir la prueba con la placa y actualizar la captura de la página.

Alimentación del servomotor. En el armado de las pruebas (Figura 6) el servo se alimentó desde el pin de 5 V del Arduino. Sin carga el sistema funcionó, pero el MG996R puede demandar corrientes de más de un ampere al arrancar o al sostener carga, y hasta unos 2.5 A con el eje bloqueado [4]. Un puerto USB normalmente entrega hasta 500 mA, por lo que esa conexión puede provocar reinicios de la placa, pérdida de la red WiFi o movimientos erráticos. Por ello el diagrama de la práctica usa una fuente externa de 4.8 a 6 V para el servo, con la tierra unida al GND del Arduino; sin esa tierra común, el pulso de control no tendría una referencia válida y el servo no respondería.

VIII. Conclusiones

Conclusión general. Se cumplió el objetivo de controlar un servomotor desde una interfaz web. El Arduino UNO R4 WiFi funcionó como punto de acceso y servidor HTTP sin requerir otra red ni aplicaciones adicionales, y la librería Servo tradujo cada ángulo en el ancho de pulso adecuado. El registro del monitor serie permitió detectar un error que no era visible en el movimiento del servo, y su corrección, junto con el envío en tiempo real, hizo el sistema más confiable y fluido. La práctica también mostró que un servo de alto par necesita una fuente externa con tierra común.

Conclusión individual (L231000103). En esta práctica aprendí a usar el Arduino UNO R4 WiFi como servidor web y a relacionar lo que pasa en la página con lo que hace el servo: cada movimiento del control se convierte en una petición HTTP y esa petición en un ancho de pulso en el pin D9. Lo que más me ayudó fue revisar el monitor serie, porque ahí noté que cada ángulo aparecía dos veces. Al investigar entendí que el navegador hace más peticiones de las que uno ve, como la del ícono, y que el programa debe leer solo la línea de la petición. También me quedó claro que un servo como el MG996R no debe alimentarse del pin de 5 V de la placa, sino de una fuente externa con la tierra unida al Arduino. Como mejora, agregaría botones con posiciones fijas y la opción de conectar el Arduino a una red existente para controlarlo desde más dispositivos.

Referencias

- [1] Arduino, «UNO R4 WiFi», documentación oficial. Disponible en:
<https://docs.arduino.cc/hardware/uno-r4-wifi>
- [2] Arduino, «Servo library», repositorio oficial. Disponible en:
<https://github.com/arduino-libraries/Servo>
- [3] Arduino, «ArduinoCore-renesas: librería Wi-FiS3», repositorio oficial. Disponible en:
<https://github.com/arduino/ArduinoCore-renesas>
- [4] Tower Pro, «MG996R High Torque Metal Gear Dual Ball Bearing Servo», hoja de datos.
- [5] MDN Web Docs, «Fetch API». Disponible en:
https://developer.mozilla.org/es/docs/Web/API/Fetch_API
- [6] R. Fielding, M. Nottingham y J. Reschke, «HTTP Semantics», RFC 9110, IETF, 2022.
- [7] Fritzing, «Fritzing: electronics made easy». Disponible en: <https://fritzing.org>